PROBLÈME · ANNALES SPÉ · 3 HEURES

Relations d'ordre, chaînes et antichaînes

1. Relation d'ordre et tri topologique

Langage OCaml exclusivement. On représente une relation sur $V = \{v_0, \dots, v_{n-1}\}$ par une matrice `r : bool array array` de taille $n \times n$, avec `r.(i).(j)` vrai si et seulement si $v_i \trianglelefteq v_j$ (`type order = bool array array`).

1. Écrire `check_reflexivity (r:order) : bool` dans le paradigme **impératif**.
2. Écrire `check_reflexivity_bis (r:order) : bool` dans le paradigme **fonctionnel**.
3. Écrire `check_antisymmetry (r:order) : bool`.
4. Écrire `check_transitivity (r:order) : bool`.
5. Écrire `is_order (r:order) : bool` à partir de 1, 3 et 4.

Le graphe de l'ordre a pour arcs les (v_i, v_j) tels que $v_i \trianglelefteq v_j$; le degré sortant d'un sommet est le nombre d'arcs qui en partent.

1. Écrire `outdegrees (r:order) : int array` (degrés sortants).
2. Écrire `argmin (d:int array) : int` (un indice d'une valeur minimale).

Un tri topologique est une permutation σ de $[\![0, n-1]\!]$ telle que $v_i \trianglelefteq v_j$ entraîne $\sigma(i) \leqslant \sigma(j)$; v_i est alors le sommet de numéro $\sigma(i)$.

1. Montrer qu'il existe une constante δ_0 telle que, pour tout tri topologique σ, le sommet de numéro $n-1$ est de degré sortant δ_0 et les autres de degré sortant $\geqslant \delta_0$.
2. Écrire `topological_sort (r:order) : int array` qui attribue les numéros par ordre décroissant en s'appuyant sur la question 8.
3. Énoncer des invariants qui prouvent la correction de `topological_sort`.
4. Complexité en temps de `topological_sort` en fonction de n ; comment la qualifier par rapport à la taille de l'entrée ?

2. Chaînes, antichaînes et tas

Langage OCaml exclusivement. On représente une relation sur $V = \{v_0, \dots, v_{n-1}\}$ par une matrice `r : bool array array` de taille $n \times n$, avec `r.(i).(j)` vrai si et seulement si $v_i \trianglelefteq v_j$ (`type order = bool array array`).

Une chaîne est une partie C telle que deux éléments quelconques c, c' vérifient $c \trianglelefteq c'$ ou $c' \trianglelefteq c$ (représentée par une `int list`). Code donné :

```
let rec is_chain (r:order) (c:int list) : bool =
  match c with
  | [] -> true
  | v::cc -> List.for_all (fun x -> x > v || x < v) cc
             && is_chain r cc
```

1. Confirmer ou réfuter : `is_chain r c` teste si `c` est une chaîne pour `r`. Dans le second cas, corriger la fonction.
2. Complexité de `is_chain` en fonction de la longueur γ de `c`.

On veut `is_chain_bis` en $O(\gamma \log \gamma)$ avec une structure mutable de capacité bornée : `init : order -> int -> hp`, `is_empty : hp -> bool`, `push : hp -> int -> bool`, `pop : hp -> bool` ; `push` et `pop` peuvent échouer, ce que signale leur valeur de retour. La fonction insère tous les éléments de `c` puis les retire tous, et renvoie la conjonction des succès.

1. Compléter la description du type `hp` en nommant une structure de données (avec son invariant) pour que `is_chain_bis` respecte la spécification en $O(\gamma \log \gamma)$. Justifier.

Une antichaîne est une partie dont les éléments sont deux à deux incomparables.

1. Modifier `is_chain` en `is_antichain` ; en déduire sa complexité.
2. Confirmer ou réfuter l'existence d'une fonction `is_antichain_bis` en $O(\alpha \log^2 \alpha)$, où α est la longueur de la liste.

3. Couverture minimale par des chaînes

Langage OCaml exclusivement. On représente une relation sur $V = \{v_0, \dots, v_{n-1}\}$ par une matrice `r : bool array array` de taille $n \times n$, avec `r.(i).(j)` vrai si et seulement si $v_i \trianglelefteq v_j$ (`type order = bool array array`).

Une couverture est une famille $(C_\ell)_{1 \leqslant \ell \leqslant \lambda}$ de chaînes dont la réunion est V.

1. Si (C_ℓ) est une couverture et A une antichaîne de cardinal α, montrer que $\alpha \leqslant \lambda$.

Graphe des poursuites : graphe biparti P sur $W_0 = \{w_i\}$ et $W_1 = \{w_{n+i}\}$ (deux copies de V), avec une arête (w_i, w_{n+j}) dès que $i \neq j$ et $v_i \trianglelefteq v_j$. Représentation : `type bipartite = int list array` de longueur $2n$, chaque arête notée deux fois. Un couplage est un ensemble d'arêtes d'extrémités distinctes.

1. Écrire `bipartite_of_order (r:order) : bipartite`.
2. Pour un couplage M, on prend la partition (C_ℓ) de V la plus fine possible telle que $(w_i, w_{n+j}) \in M$ impose que v_i et v_j soient dans la même part. Montrer que chaque C_ℓ est une chaîne.

Un couplage est codé par `m : int array` de longueur n : `m.(j) = i` si $(w_i, w_{n+j}) \in M$, et -1 si w_{n+j} n'est pas couvert.

1. Écrire `chains_of_matching (m:int array) : int list list`.
2. Une couverture par des chaînes disjointes étant fixée, construire un couplage qui la redonne par la question 19.
3. Montrer que la construction de 19 donne une couverture par un nombre minimum de chaînes si et seulement si M est de cardinal maximum.
4. Écrire `maximum_matching (p:bipartite) : int array`.
5. Écrire `minimum_cover (r:order) : int list list`.
6. Complexité de `minimum_cover` en fonction de n.